Taming the Garbage Collector – how the worst enemy of the programmers change into the best friend

Beata Zalewa, 4Developers Gdańsk, 25.09.2018

zalnet

About me

Security Architect

Consultant

Microsoft Certified Trainer

AI & Cybersecurity Practitioner

Developer

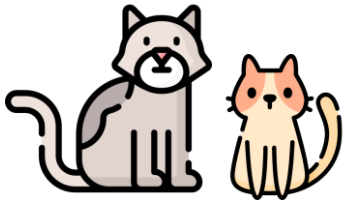
Freelancer

Azure @ ❤

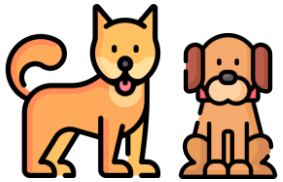
Google Cloud

1 Husband

1 Daughter



2 cats



2 dogs

Detective stories

Photography

zalnet BEATA ZALEWA

https://www.linkedin.com/in/beatazalewa/

info@zalnet.pl

BLOG https://zalnet.pl/

AGENDA

- Managed and unmanaged resources vs Garbage Collector
- Garbage Collection
- .NET Memory Management
- BenchmarkDotNet package
- Working with LINQ
- MemoryDiagnoser
- Memory Leaks
- Roslyn-based heap allocation analyzer

zalnet

STEP I

Managed and unmanaged resources vs Garbage Collector

zalnet

MANAGED RESOURCES

- Managed Resources are those that are pure .NET code.
- They are managed by the runtime and are under its direct control. e.g. int, long, double, string, struct or class.
- While programming in the managed environment, you allocate memory on the managed heap using the new operator:

```
var obj = new ClassName();
```

- When your work is done, then the garbage collector (GC) reclaims that memory.

zalnet

UNMANAGED RESOURCES

- Unmanaged Resources are those that are not under direct control of runtime:

- Database Connections
- File
- File streams
- COM objects
- etc.

zalnet

GARBAGE COLLECTOR

- Garbage Collector is responsible for freeing memory which is no longer required.
- GC knows only about Managed Resources.
- At some undeterministic point in time, the GC will come along and clean up all the memory and resources associated with a managed object.

zalnet

WHO'LL TAKE CARE OF UNMANAGED RESOURCES??

- Finalizer and IDisposable
- If your class wraps any unmanaged resource, then you should have a Finalizer and implement IDisposable.
- IDisposable interface has a method Dispose(), used to clean unmanaged resources explicitly (you explicitly call this method for cleaning).
- Finalizer is also used to clean all unmanaged resources, but not explicitly, so you write the code of cleaning but you do not explicitly call this method. It will be called by GC.
- Finalizer is basically a safegaurd if you forget to do cleaning explicitly via IDisposable.
- Since both IDisposable and Finalizer serve the same purpose of cleaning unmanaged resources, better we keep core cleaning logic in a single place.

zalnet

IDISPOSABLE INTERFACE

```
class SomeClass : IDisposable
{
    // Flag: Has Dispose already been called?
    bool disposed = false;

    // Public implementation of Dispose pattern callable by consumers.
    0 references
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
    // Protected implementation of Dispose pattern.
    2 references
    protected virtual void Dispose(bool disposing)
    {
        if (disposed)
            return;

        if (disposing)
        {
            // Free any other managed objects here.
        }
        // Free any unmanaged objects here.
        disposed = true;
    }
    0 references
    ~SomeClass()
    {
        Dispose(false);
    }
}
```

zalnet

DEMO

Garbage Collection Demo

zalnet

STEP II

Garbage Collection

zalnet

HOW GARBAGE COLLECTOR WORKS?

- The purpose of the garbage collector is to discover objects on the managed heap that are no longer reachable and to reclaim the memory occupied by these objects.
- Once an object is discovered to be no longer reachable, the garbage collector checks to see if the object has a finalizer.
- If the object doesn't have a finalizer, it is marked for collection and its memory is reclaimed by the garbage collector.

HOW GARBAGE COLLECTOR WORKS?

- If an object has a finalizer, it will not be marked for garbage collection, but instead be marked for finalization.
- It will not get garbage collected during the current pass of the garbage collector.
- Instead, its finalizer will be run at some point after garbage collection finishes. It will then be eligible for collection during the **next** pass of the garbage collector.
- Reclaiming memory for objects with finalizers therefore requires at least two passes of the garbage collector.

OBJECTS WITH FINALIZERS

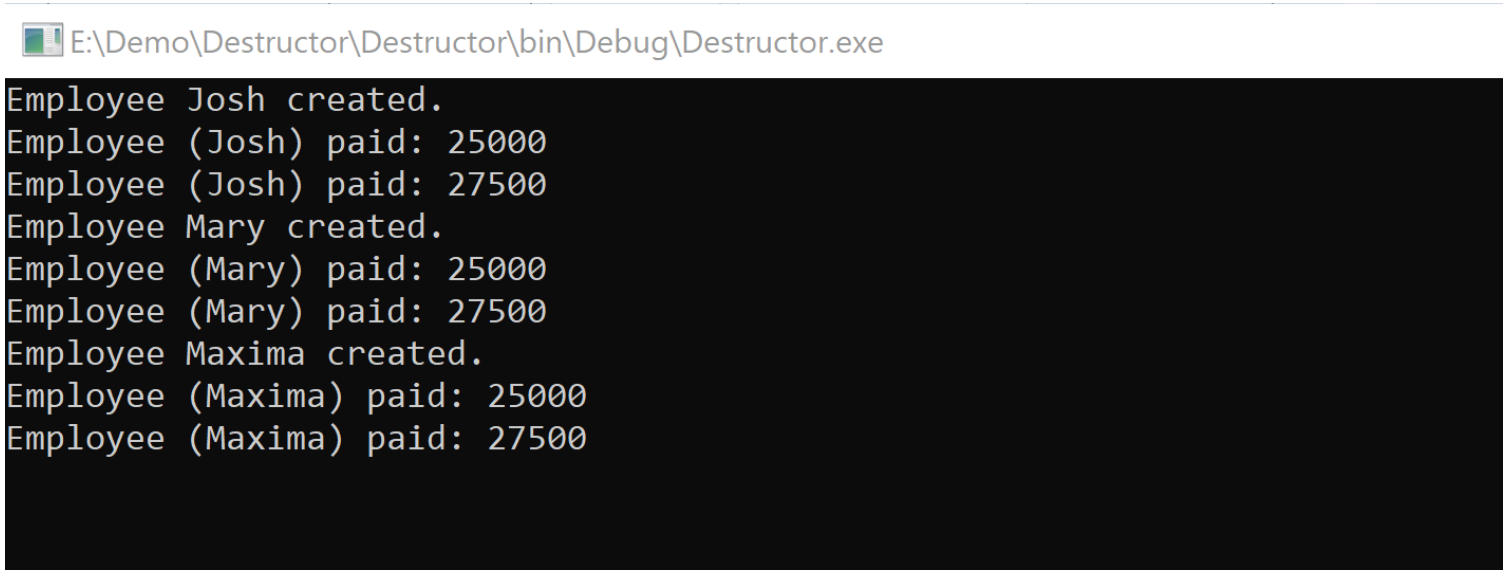
- In general an object can be garbage collected when it is no longer referenced by any other object or reference-typed variable.
- For example, the object named „Josh" is no longer referenced in the code below when the josh variable is set to null.

```
static void Main(string[] args)
{
    Employee josh = new Employee("Josh");
    josh.PaySalary();
    josh.PayRise();
    josh.PaySalary();
    //Josh no longer exists after this line executes
    josh = null;
    Employee mary = new Employee("Mary");
    mary.PaySalary();
    mary.PayRise();
    mary.PaySalary();
    mary = null;
```

zalnet

OBJECTS WITH FINALIZERS

- In practice, the situation can be different:



zalnet

DEMO

Destructor

zalnet

GENERATIONS

- Referenced-typed objects in an application have different lifetimes.
- The application will use some objects as long as the application is running. Others are referenced only during execution of a single method.
- If the garbage collector always examined every object whenever it did a garbage collection pass, it would spend a lot of time reexamining longer-living objects that can't yet be garbage collected.
- The garbage collector can perform more efficiently by looking at only a subset of all objects during each pass. It does this by grouping objects into ***generations***.

GENERATIONS

- **Generations:**
- Generation 0 – Objects that have been created since the last GC pass *(newest objects)*
- Generation 1 – Objects that have survived one pass of the GC
- Generation 2 – All other objects *(oldest objects)*
- The garbage collector examines and collect objects in generation 0, moving to higher generations only if it needs additional memory.
- Objects are promoted to the next generation only if a GC pass is done on the generation in which they are located.

GENERATIONS

- When the garbage collector performs a GC pass, it looks only at objects in generation 0, releasing memory for any that are no longer reachable.
- In this way, the garbage collector is more efficient, because it is only looking at a portion of the managed heap.
- If the garbage collector has collected Gen 0, but the application requires more memory, the garbage collector can then garbage collect generation 1.
- If the application still requires memory after collecting generation 1, it can move on to generation 2.
- This scheme relies on the fact that if an object survives one GC pass, it likely has a longer lifetime and will survive future passes.

GENERATIONS

- Objects are only promoted to the next generation if the generation that they are currently located in is examined and collected during a garbage collection pass.

- **This means:**

1. Since the GC always examines Gen 0 during any GC pass, Gen 0 objects that survive a garbage collection are always promoted to Gen 1.
2. Objects in Gen 1 are only promoted to Gen 2 if Gen 1 is examined and collected during a GC pass and the object survives.
3. The GC will very often do only a Gen 0 pass during collection.
4. When the GC only examines and collects Gen 0, Gen 1 objects are not examined and therefore not promoted to Gen 2.

LARGE OBJECTS ARE ALLOCATED ON THE LARGE OBJECT HEAP

- The managed heap is the area in memory where reference-typed objects are allocated.
- When you create a new object, a portion of the managed heap is allocated for the object.
- In reality, objects are stored on either the Small Object Heap (SOH) or the Large Object Heap (LOH).
- Objects that are larger than 85,000 bytes are allocated on the LOH. All other objects are allocated on the SOH.
- This improves the performance of the garbage collector (GC), since it typically collects objects in Generation 0 (newest), only moving to older generations if necessary.

LARGE OBJECTS ARE ALLOCATED ON THE LARGE OBJECT HEAP

- **Other differences between the two heaps:**

1. The SOH is generational–objects belong to generation 0, 1 or 2.
2. The LOH is not sub-divided into generations.
3. After the LOH is garbage collected, the heap is not compacted. This results in the memory becoming fragmented and requires maintaining a linked list of free blocks. (The SOH is compacted after every collection).
4. Allocation on the LOH can be slower than the SOH, due to the fragmentation.

FORCING A GARBAGE COLLECTION

- The garbage collector (GC) normally runs automatically, doing a garbage collection pass when necessary (when there is memory pressure and you are allocating memory for some new object).
- You typically just let the garbage collector run automatically, never explicitly asking it to do garbage collection.
- For testing purposes, however, you might want to force garbage collection to happen at a particular time. You can do this by calling the **GC.Collect** method.
- Calling **Collect** will force a collection across all generations. You can also specify the highest generation to collect as follows:

1. **GC.Collect()** – Collect generations 0, 1, 2
2. **GC.Collect(0)** – Collect generation 0 only
3. **GC.Collect(1)** – Collect generations 0, 1

zalnet

FORCING A GARBAGE COLLECTION

- Objects with finalizers will not be collected when you call **Collect**, but rather placed on the finalization queue.
- If you want to also release memory for these objects, you need to wait until their finalizers are called and then do another garbage collection pass.

```
1  GC.Collect();
2  GC.WaitForPendingFinalizers();
3  GC.Collect();
```

zalnet

FORCING A GARBAGE COLLECTION

- The garbage collector (GC) groups objects into generations to avoid having to examine and collect all objects in memory whenever a garbage collection is done.
- For debugging purposes, it's sometimes useful to know which generation an object currently belongs to. You can get this information using the **GC.GetGeneration** method.

```
Cat myCousinCat = new Cat("Garfield", 5);
Console.WriteLine(string.Format("Garfield is in generation {0}",
   GC.GetGeneration(myCousinCat)));

GC.Collect();
Console.WriteLine(string.Format("Garfield is in generation {0}",
   GC.GetGeneration(myCousinCat)));

GC.Collect();
Console.WriteLine(string.Format("Garfield is in generation {0}",
   GC.GetGeneration(myCousinCat)));
```

zalnet

FORCING A GARBAGE COLLECTION

- In the example above, the "Garfield" **Cat** object starts out in generation 0.
- After we do the first garbage collection, it's promoted to generation 1 and after the 2nd collection, it's promoted to generation 2.

```
Garfield is in generation 0
Garfield is in generation 1
Garfield is in generation 2
```

zalnet

FINALIZERS ARE CALLED WHEN AN APPLICATION SHUTS DOWN

- A finalizer is an override of the **System.Object.Finalize** method that you implement using the destructor syntax.
- The finalizer is called automatically by the garbage collector, before it releases memory for the object.
- Finalizers are also called when an application shuts down.
- Before the application shuts down, the finalizers of any objects that still exist on the managed heap are called.
- This is only true if the application shuts down in a controlled manner. If the application crashes, the finalizers will not be called.

CHECKING TO SEE IF OBJECTS ARE DISPOSABLE

- If a type implements the **IDisposable** interface, you should always call the **Dispose** method on an instance of the class when you are done using it. The presence of **IDisposable** indicates that the class has some resources that can be released prior to garbage collection.
- One pattern for checking if an object implements the **IDisposable** interface.

```
Cat myDaughterCat = new Cat("Felix", 8);
myDaughterCat.Purr();
if (myDaughterCat is IDisposable)
    ((IDisposable)myDaughterCat).Dispose();
```

CHECKING TO SEE IF OBJECTS ARE DISPOSABLE

- You can also use the **using** statement to automatically call **Dispose** on an object when you're done using it.
- In this case class must implement IDisposable interface.

```
using (Cat myHusbandCat = new Cat("Fiona", 4))
{
    myHusbandCat.Purr();
}
```

zalnet

DEMO

Cats Demo

zalnet

STEP III

.NET Memory Management

zalnet

BASIC LEVEL OF MEMORY MANAGEMENT

To perform efficient memory manipulation in performance critical system, we need to have a good understanding of memory management.

zalnet

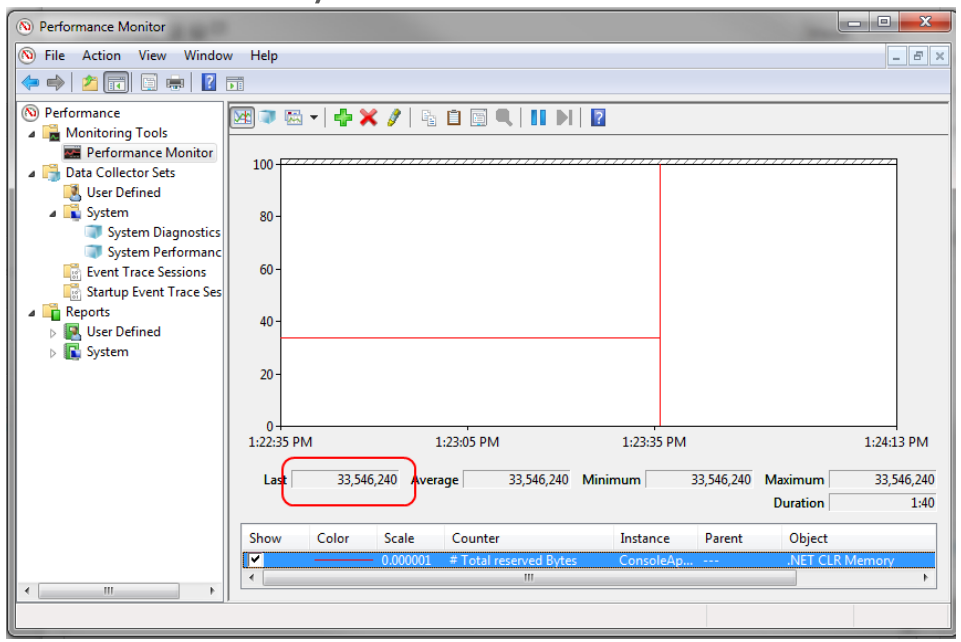
.NET MEMORY PERFORMANCE COUNTERS UPDATED WHEN GARBAGE COLLECTOR RUNS

- The .NET CLR Memory performance counters can be used to track memory usage for CLR-based (.NET) applications.
- The counters, however, are only updated when the garbage collector runs. This means that if you're watching one of the counters, you won't necessarily see it update immediately after your application allocates some memory.
- If you are doing some debugging or profiling related to memory management, you can force garbage collection at the point where you want to measure memory usage.
- You do this using the **GC.Collect** method. **GC.Collect** should not typically be called explicitly in production code.

zalnet

LOOKING AT .NET MEMORY USAGE USING PERFORMANCE COUNTERS

- Provide numeric performance information about the system
- Located in different areas in the system – disk, .NET, networking, OS objects
- Available on-demand using built-in tools like *perfmon* and *typeperf*
- You can write your own



zalnet

LOOKING AT .NET MEMORY USAGE USING PERFORMANCE COUNTERS

- As you create new objects in .NET, the garbage collector allocates memory for the objects on the heap. As memory is allocated, the process consumes some of its available virtual memory. (In practice, a 32-bit process can allocate at most around 1.5GB of its 2GB maximum).
- You can keep track of (roughly) how much memory on the managed heap that your application has allocated by using performance counters.

zalnet

EVENT TRACING FOR WINDOWS

- High-speed logging framework supporting more than 100K structured messages per second
- ▸ .NET, drivers, services, third party components
- ▸ Can be turned on on-demand while running
- ▸ Very small overhead

zalnet

LOOKING AT .NET MEMORY USAGE USING PERFORMANCE COUNTERS

To track memory usage:

1. Start Performance Monitor.
2. Start your application.
3. Click the "+" icon to add a counter.
4. Expand the **.NET CLR Memory** section.
5. Select the **# Total Reserved Bytes** counter and select your application.
6. Click the **Add** button and then click **OK.**

- You'll now see a graph indicating total number of bytes of memory that your application has reserved. The **Last** field will show you the most recent value.

zalnet

DEMO

Get TotalMemory indicates how much memory you've allocated

zalnet

STEP IV

BenchmarkDotNet package

zalnet

COMMON PROGRAMMING CHALLENGE

- A common programming challenge is how to manage complexity around code performance – a small change might have a large impact on application performance.

- Sometimes you want to be able to perform micro-benchmarks against your code before and after and see really small changes, and know right away if you've made things better or worse.
- BenchmarkDotNet helps in this situation.

zalnet

HOW TO START?

Project with installed BenchmarkDotNet package and class witch Benchmark attribute:

```
using System;
using BenchmarkDotNet.Attributes;

namespace Services
{
    0 references
    public class RandomNumberGenerator : IRandomNumberGenerator
    {
        private static Random random = new Random();

        [Benchmark]
        2 references | 0 exceptions
        public int GetRandomNumber()
        {
            return random.Next();
        }

    }
}
```

zalnet

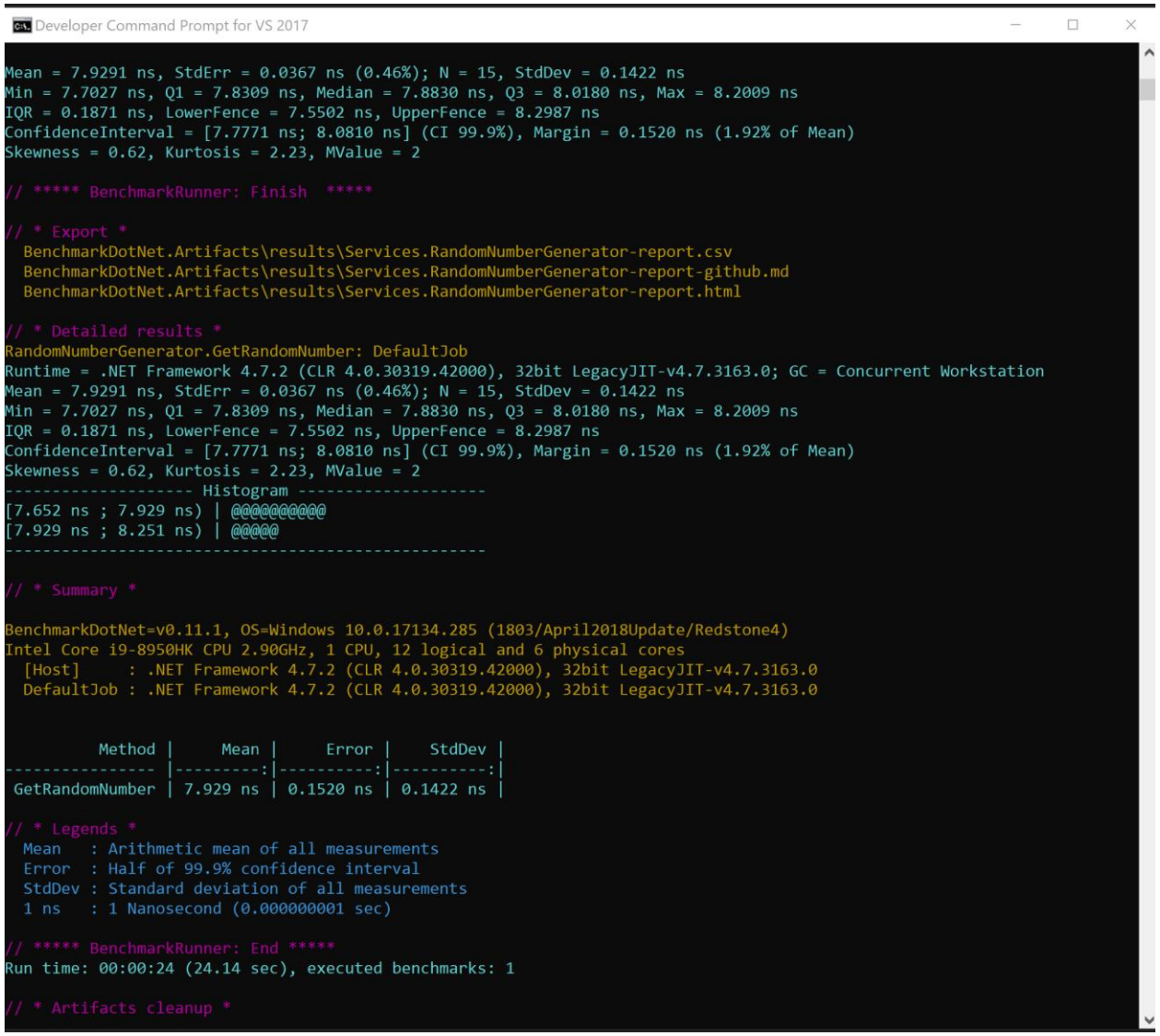
HOW TO START?

Call the BenchmarkRunner method:

```
using BenchmarkDotNet.Running;
using Services;


namespace PerformanceRunner
{
    0 references
    class Program
    {
        0 references | 0 exceptions
        static void Main(string[] args)
        {
            var summary = BenchmarkRunner.Run<RandomNumberGenerator>();
        }
    }
}
```

zalnet

RESULTS



zalnet

RESULTS

Developer Command Prompt for VS 2017

```
// * Summary *

BenchmarkDotNet=v0.11.1, OS=Windows 10.0.17134.285 (1803/April2018Update/Redstone4)
Intel Core i9-8950HK CPU 2.90GHz (Max: 1.50GHz), 1 CPU, 12 logical and 6 physical cores
  [Host]     : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0
  DefaultJob : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0


 Method | Number |      Mean |     Error |    StdDev |
------- |------- |----------:|----------:|----------:|
 Square |      1 | 0.0168 ns | 0.0217 ns | 0.0275 ns |
 Square |      2 | 0.0500 ns | 0.0386 ns | 0.0502 ns |

// * Hints *
Outliers
  MathFunctions.Square: Default -> 2 outliers were removed
  MathFunctions.Square: Default -> 1 outlier  was  removed

// * Legends *
  Number : Value of the 'Number' parameter
  Mean   : Arithmetic mean of all measurements
  Error  : Half of 99.9% confidence interval
  StdDev : Standard deviation of all measurements
  1 ns   : 1 Nanosecond (0.000000001 sec)

// ***** BenchmarkRunner: End *****
Run time: 00:01:58 (118.26 sec), executed benchmarks: 2

// * Artifacts cleanup *
```

zalnet

DEMO

Benchmark Demo

zalnet

STEP V

Working with LINQ

zalnet

LINQ

- Language-Integrated Query (LINQ) is the name for a set of technologies based on the integration of query capabilities directly into the C# language.
- Traditionally, queries against data are expressed as simple strings without type checking at compile time or IntelliSense support. Furthermore, you have to learn a different query language for each type of data source: SQL databases, XML documents, various Web services, and so on.
- With LINQ, a query is a first-class language construct, just like classes, methods, events.

WHICH WAY IS FASTER?

Is a LINQ statement faster than a 'foreach' loop?

90

I am writing a Mesh Rendering manager and thought it would be a good idea to group all of the meshes which use the same shader and then render these while I'm in that shader pass.

I am currently using a `foreach` loop, but wondered if utilising LINQ might give me a performance increase?

15

c# performance linq foreach

share improve this question

edited Aug 29 '16 at 15:29

Stijn
15.3k ● 9 ● 74 ● 125

asked Jul 1 '10 at 8:18

Neil Knight
35.2k ● 19 ● 104 ● 175

1 possible duplicate of "Nested foreach" vs "lambda/linq query" performance(LINQ-to-Objects) – Daniel Earwicker Jul 1 '10 at 8:22

1 Please consider setting @MarcGravell's answer to the accepted one, there are situations, linq to sql for example, where linq is faster than the for/foreach. – paqogomez Oct 10 '14 at 15:50

add a comment

8 Answers

active oldest votes

163

Why should LINQ be faster? It also uses loops internally.

Most of the times, LINQ will be a bit slower because it introduces overhead. Do not use LINQ if you care much about performance. Use LINQ because you want shorter better readable and maintainable code.

zalnet

HOW TO CHECK IT?

Compare 3 different queries with the same end result:

```
P:\Konferencje\4Developers2018\GarbageCollector\Demo\WorkingWithLINQ\WorkingWithLINQ\bin\Debug>Wor
3,54294
3,54294
3,54294
```

```
// * Summary *

BenchmarkDotNet=v0.11.1, OS=Windows 10.0.17134.285 (1803/April2018Update/Redstone4)
Intel Core i9-8950HK CPU 2.90GHz, 1 CPU, 12 logical and 6 physical cores
  [Host]     : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0
  DefaultJob : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0


                    Method |      Mean |    Error |   StdDev |
-------------------------- |----------:|---------:|---------:|
          SumUsingForLoops |  69.42 ms | 1.385 ms | 3.154 ms |
 SumUsingForLoopsAndString | 105.06 ms | 1.213 ms | 1.134 ms |
             SumUsingLinq | 597.48 ms | 7.652 ms | 7.157 ms |
```

zalnet

DEMO

Working with LINQ

zalnet

STEP VI

MemoryDiagnoser

zalnet

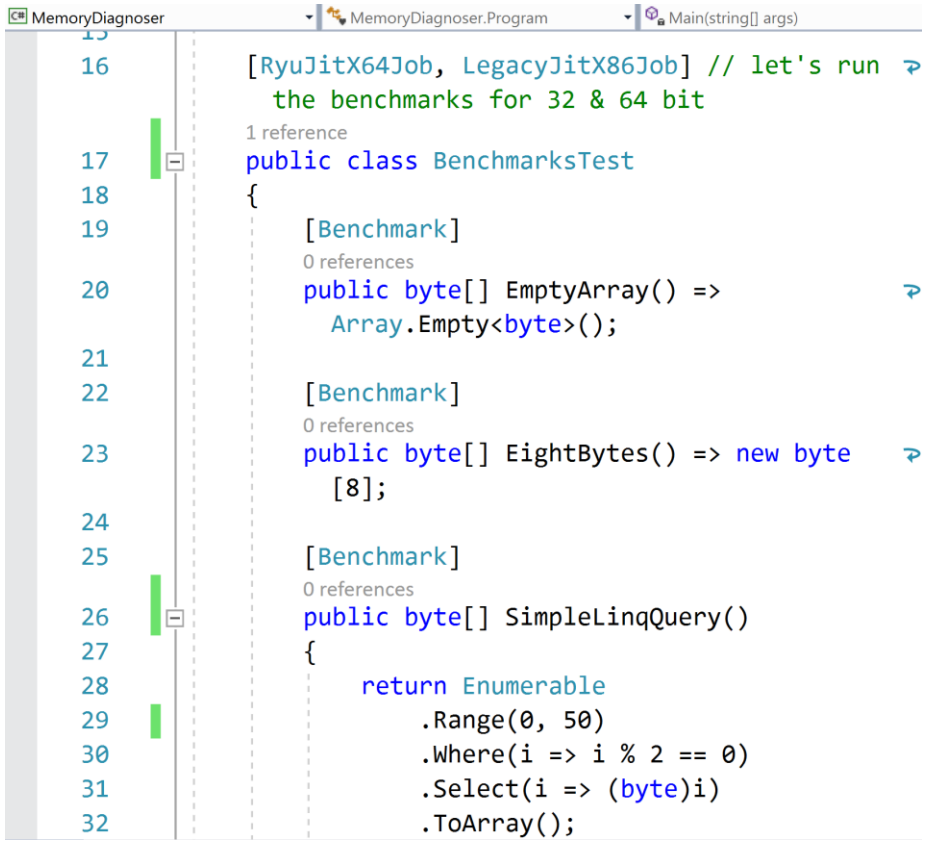
MEMORY DIAGNOSER

- MemoryDiagnoser is one of its features that allows measuring the number of allocated bytes and garbage collection frequency.
- Before the **0.10.1** version of BenchmarkDotNet the MemoryDiagnoser was part of **BenchmarkDotNet.Diagnostics.Windows** package. Internally it was using Event Tracing for Windows (ETW), which had following implications:
- It was not cross-platform (**Windows only**).
- It was as accurate as ETW allowed us it to be. We were using the *GCAllocationTick* event which is raised *"Each time approximately 100 KB is allocated"*. Which means that if 199 KB was allocated we would know only about the first 100 KB.

zalnet

MEMORY DIAGNOSER

- Class with Benchmark attribute:



zalnet

MEMORY DIAGNOSER

Developer Command Prompt for VS 2017

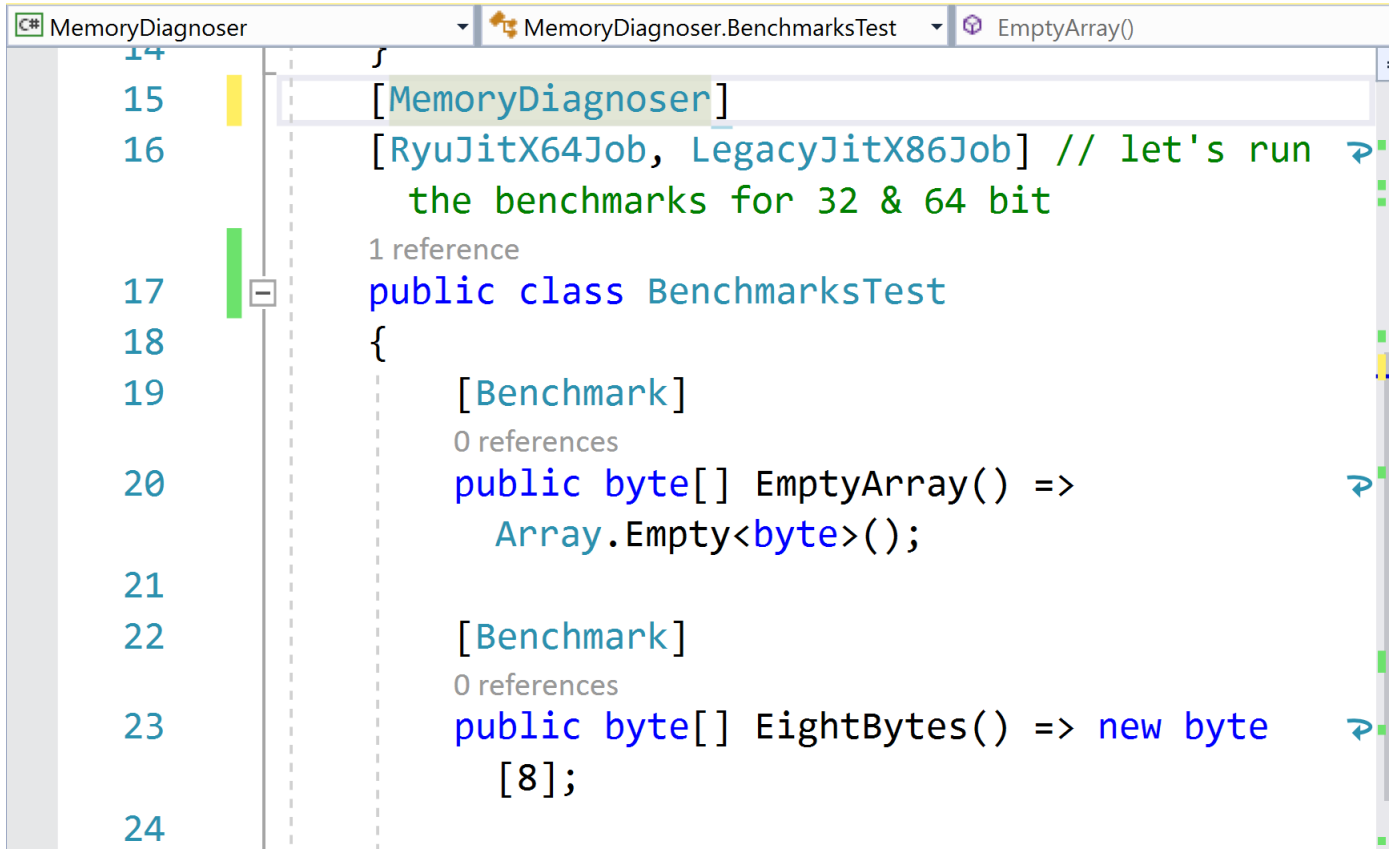
```
// * Summary *

BenchmarkDotNet=v0.11.1, OS=Windows 10.0.17134.285 (1803/April2018Update/Redstone4)
Intel Core i9-8950HK CPU 2.90GHz, 1 CPU, 12 logical and 6 physical cores
  [Host]       : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0
  LegacyJitX86 : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 32bit LegacyJIT-v4.7.3163.0
  RyuJitX64    : .NET Framework 4.7.2 (CLR 4.0.30319.42000), 64bit RyuJIT-v4.7.3163.0

Runtime=Clr

         Method |          Job |       Jit | Platform |        Mean |      Error |     StdDev |      Median |
--------------- |------------- |---------- |--------- |------------:|-----------:|-----------:|------------:|
     EmptyArray | LegacyJitX86 | LegacyJit |      X86 |   0.1744 ns |  0.0588 ns |  0.1340 ns |   0.1745 ns |
     EightBytes | LegacyJitX86 | LegacyJit |      X86 |   2.3960 ns |  0.1246 ns |  0.2962 ns |   2.2990 ns |
 SimpleLinqQuery | LegacyJitX86 | LegacyJit |      X86 | 755.0748 ns | 10.2032 ns |  9.5441 ns | 758.3945 ns |
     EmptyArray |    RyuJitX64 |    RyuJit |      X64 |   0.0031 ns |  0.0110 ns |  0.0113 ns |   0.0000 ns |
     EightBytes |    RyuJitX64 |    RyuJit |      X64 |   3.0174 ns |  0.1499 ns |  0.4372 ns |   2.9502 ns |
 SimpleLinqQuery |    RyuJitX64 |    RyuJit |      X64 | 669.3525 ns | 13.1610 ns | 25.6694 ns | 662.4184 ns |

// * Warnings *
MultimodalDistribution
  BenchmarksTest.EmptyArray: LegacyJitX86 -> It seems that the distribution is bimodal (mValue = 3.88)
  BenchmarksTest.EightBytes: RyuJitX64    -> It seems that the distribution is multimodal (mValue = 4.64)

// * Hints *
Outliers
  BenchmarksTest.EightBytes: LegacyJitX86   -> 2 outliers were removed
  BenchmarksTest.SimpleLinqQuery: RyuJitX64 -> 3 outliers were removed

// * Legends *
  Mean   : Arithmetic mean of all measurements
  Error  : Half of 99.9% confidence interval
  StdDev : Standard deviation of all measurements
  Median : Value separating the higher half of all measurements (50th percentile)
  1 ns   : 1 Nanosecond (0.000000001 sec)

// ***** BenchmarkRunner: End *****
Run time: 00:06:08 (368.12 sec), executed benchmarks: 6

// * Artifacts cleanup *

E:\Demo\MemoryDiagnoser\MemoryDiagnoser\bin\Release>
```

zalnet

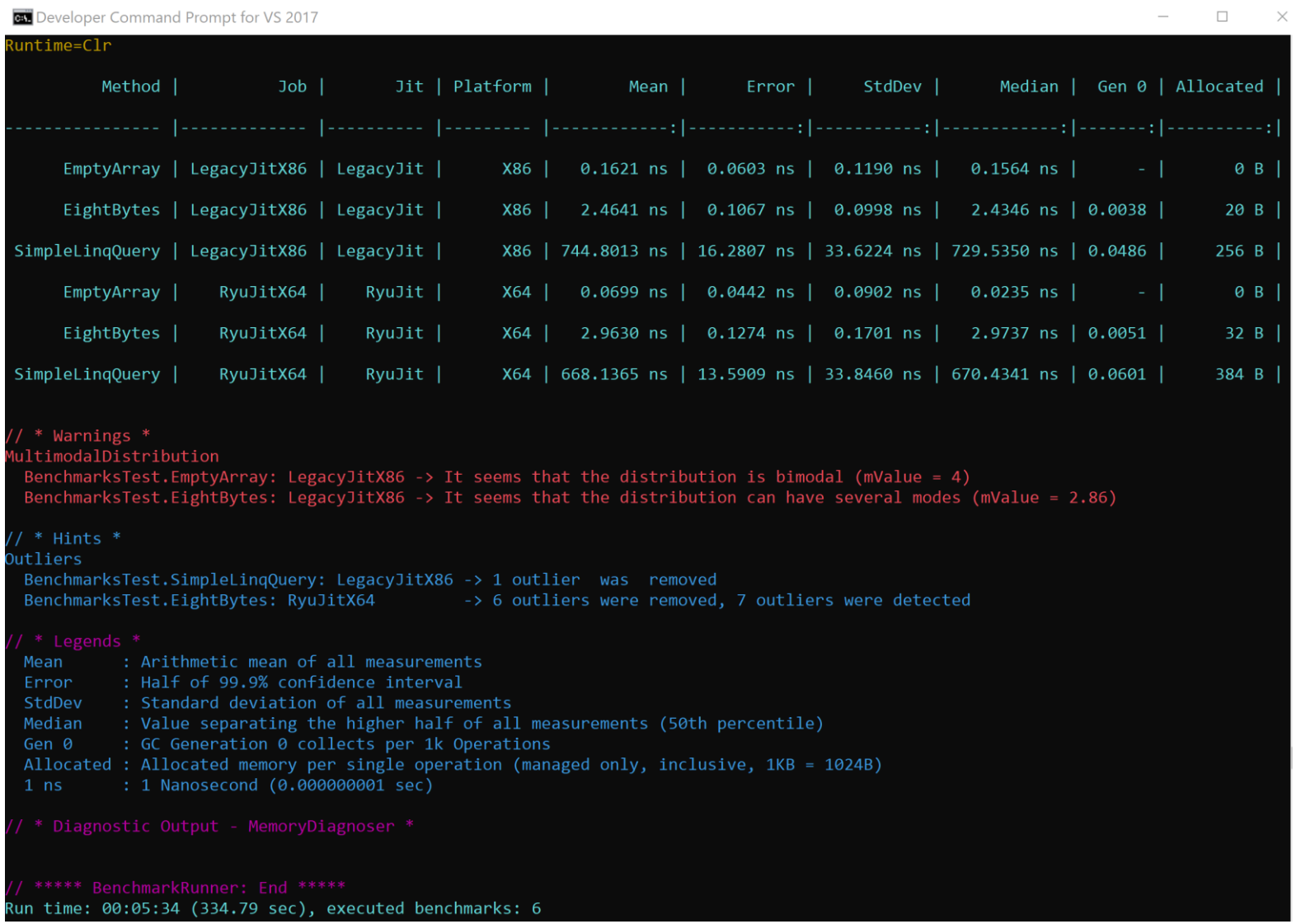
Email: beata@zalnet.pl www.zalnet.pl

MEMORY DIAGNOSER

- Add the MemoryDiagnoser attribute to the class :



MEMORY DIAGNOSER



zalnet

Email: beata@zalnet.pl www.zalnet.pl

DEMO

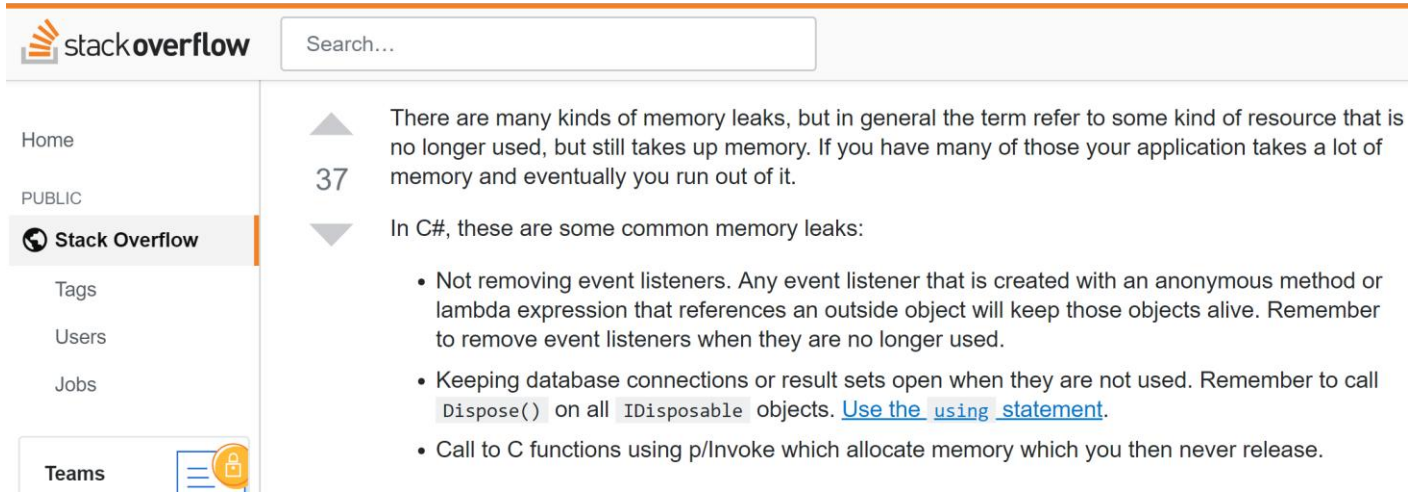
MemoryDiagnoser

STEP VII

Memory Leaks

zalnet

WHAT IS MEMORY LEAK?



zalnet

DEMO

Issues with Memory Leaks

zalnet

STEP VIII

Roslyn-based heap allocation analyzer

zalnet

CLR HEAP ALLOCATION ANALYZER

- Roslyn based C# heap allocation diagnostic analyzer.
- It can detect most heap allocations including explicit allocations, value type to reference type (boxing), closure captures (a.k.a Display Classes) and can tell you why the closure is being captured. Implicit delegate creation and implicit allocations done by the compiler for params, etc.
- It can also run as part of your build and flag as warnings. It is, however, most demonstrative in its code-assist form in the IDE.
- It can be downloaded from Visual Studio MarketPlace.
- https://github.com/mjsabby/RoslynClrHeapAllocationAnalyzer for the source.

zalnet

DEMO

Heap Analyzer

zalnet

I am actively seeking new opportunities and exciting challenges. If you would like to get in touch, please feel free to reach out through the following channels:

zalnet
BEATA ZALEWA

Email: info@zalnet.pl

LinkedIn: https://www.linkedin.com/in/beatazalewa/

Blog: https://zalnet.pl/blog/

X: https://x.com/beatazalewa

GitHub: https://github.com/beatazalewa/Conferences/